

Assignment No. 02

Generative AI

Fall 2026

Deadline: As Per GCR

*Text-to-SQL with the Transformer
(built and trained from scratch)*

Submission Instructions:

- This is a group assignment.
- Each group should be exactly 2 members.
- **Plagiarism is strictly prohibited. Plagiarism among groups would be checked with tools.**
- **Please do not use any AI tool for direct copy / paste and do not copy from others.**
- **AI plagiarism using tools such as ChatGPT, Claude, Gemini, Copilot, Cursor, etc. will be marked straight zero.**
- **Submission carries no marks. Marks will be awarded based on viva.**
- Write a Medium Blog and a LinkedIn Post and submit their links.
- Develop the front end, and push all code to GitHub with commits from both team members. Submit Repo Link.
- Use Colab or Kaggle for GPU access.

Read the whole manual before you begin.

Contents

1	Overview	3
1.1	The problem	3
1.2	Constraints	3
1.3	Dataset: WikiSQL	3
1.4	How the starter code frames the task	4
2	Instructions	4
2.1	Task 1 – Run the starter code	4
2.2	Task 2 – Implement the Transformer	4
2.3	Task 3 – Train	5
2.4	Task 4 – Decode and convert to SQL	5
2.5	Task 5 – Evaluate and analyse	6
2.6	Task 6 – Front end and communication	6
2.7	Required repository layout	6
3	Evaluation	6
3.1	Official metrics	6
3.2	Correctness checks	7
4	Results	7
4.1	Table 1 – Data	7
4.2	Table 2 – Model and training	8
4.3	Table 3 – Official metrics	8
4.4	Table 4 – Component accuracy (dev)	8
4.5	Figures	8
4.6	Qualitative samples	8
	Appendix: Starter Code	9

1. Overview

1.1 The problem

Most of the world’s useful data sits in database tables, and most people who need it cannot write SQL. In this assignment you will build a model that turns an English question about a table into the SQL query that answers it.

Input	An English question and the column names of one table
Output	A SQL query over that table

Example. Table columns: *Player, No., Nationality, Position, Years in Toronto, School/Club Team*.

Question	<i>What is Terrence Ross’ nationality?</i>
SQL	<code>SELECT Nationality FROM table WHERE Player = 'terrence ross'</code>

You will implement the **full encoder–decoder Transformer** from *Attention Is All You Need* (Vaswani et al., 2017) yourself, train it once from random initialisation, and measure whether the generated queries return the correct answer when executed on the real table.

1.2 Constraints

- **One model, built from scratch.** The original Transformer, written by you from basic PyTorch layers. No pretrained weights, no ready-made Transformer modules.
- **Starter code covers the input side only.** Data loading, preprocessing, the tokenizer, the data loaders and the embedding + positional-encoding layer are given in the Appendix. Everything after the embeddings is your work.
- **Shipped.** The model runs behind a small web front end.
- **Explained.** All marks come from the viva.

1.3 Dataset: WikiSQL

WikiSQL (Zhong, Xiong and Socher, 2017): 80,654 hand-annotated English questions paired with SQL queries over 24,241 tables taken from Wikipedia. The official splits share *no* tables, so the model is always tested on tables it has never seen.

Repository (data + official evaluator)	https://github.com/salesforce/WikiSQL
Paper	https://arxiv.org/abs/1709.00103

Split	Questions	Files
Train	56,355	<code>train.jsonl, train.tables.jsonl, train.db</code>
Dev	8,421	<code>dev.jsonl, dev.tables.jsonl, dev.db</code>
Test	15,878	<code>test.jsonl, test.tables.jsonl, test.db</code>

Every query has the same shape:

`SELECT [agg] col FROM table [WHERE col op value (AND col op value)*]`

with $\text{agg} \in \{\text{none}, \text{MAX}, \text{MIN}, \text{COUNT}, \text{SUM}, \text{AVG}\}$ and $\text{op} \in \{=, >, <\}$.

1.4 How the starter code frames the task

The starter code turns each example into a plain text-to-text pair. Columns are referred to by special tokens <c0>, <c1>, ..., so the model never has to spell a column name; it only has to point at the right column.

Source	what is terrence ross' nationality <sep> <c0> player <c1> no. <c2> nationality <c3> position <c4> years in toronto <c5> school/club team
Target	select <c2> where <c0> = terrence ross

Source and target share **one** BPE vocabulary (8,000 pieces), exactly as in the paper, so the encoder embedding, the decoder embedding and the output projection can all share one weight matrix.

2. Instructions

Complete the tasks in order. Every deliverable must be in your GitHub repository.

Allowed and forbidden

Allowed: torch, nn.Linear, nn.Embedding, nn.LayerNorm, nn.Dropout, nn.ReLU, torch.softmax, torch.matmul, the starter code, and any front-end framework.

Forbidden anywhere in your code: nn.Transformer, nn.TransformerEncoder(Layer), nn.TransformerDecoder(Layer), nn.MultiheadAttention, F.scaled_dot_product_attention, the Hugging Face transformers library, and any pretrained weights.

2.1 Task 1 – Run the starter code

- 1.1 Download WikiSQL and set up the folders as in Appendix A.2.
- 1.2 Run data_prep.py, tokenizer.py and check_starter.py.
- 1.3 Report: pairs per split, mean and maximum source and target length in tokens, and the shapes printed by check_starter.py.
- 1.4 Plot the positional-encoding matrix (first 100 positions $\times$ 256 dimensions) as a heat-map and explain in two sentences what it shows.

2.2 Task 2 – Implement the Transformer

Build every component below as its own nn.Module, following Section 3 of the paper. Use **exactly** this configuration:

d_{model}	256
Heads h	4 ($d_k = d_v = 64$)
Encoder layers N / decoder layers N	3 / 3
Feed-forward inner size d_{ff}	1024
Dropout	0.1
Normalisation	post-norm, LayerNorm($x + \text{Sublayer}(x)$), as in the paper
Weight sharing	encoder embedding = decoder embedding = output projection

- 2.1 **Scaled dot-product attention:** $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$, with an optional mask. Masked positions get $-\infty$ (or a very large negative number) before the softmax. Return both the output and the attention weights.
- 2.2 **Multi-head attention:** project Q, K, V with W_i^Q, W_i^K, W_i^V , run h heads in parallel, concatenate, and project with W^O .
- 2.3 **Position-wise feed-forward:** $\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$.
- 2.4 **Masks:** a *padding mask* that hides `<pad>` tokens (encoder self-attention, decoder cross-attention, decoder self-attention) and a *causal mask* that stops decoder position t from seeing positions $> t$.
- 2.5 **Encoder layer:** self-attention $\rightarrow$ add & norm $\rightarrow$ FFN $\rightarrow$ add & norm. Stack N of them.
- 2.6 **Decoder layer:** masked self-attention $\rightarrow$ add & norm $\rightarrow$ cross-attention over the encoder output $\rightarrow$ add & norm $\rightarrow$ FFN $\rightarrow$ add & norm. Stack N of them.
- 2.7 **Full model:** starter `InputLayer` for both sides, your encoder and decoder, and a final linear layer to the vocabulary whose weight *is* the shared embedding matrix. Report the total number of trainable parameters.

2.3 Task 3 – Train

- 3.1 **Teacher forcing:** decoder input is `tgt[:, :-1]`, prediction target is `tgt[:, 1:]`.
- 3.2 **Loss:** cross-entropy with **label smoothing** $\epsilon_{ls} = 0.1$, ignoring `<pad>`.
- 3.3 **Optimiser:** Adam with $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$.
- 3.4 **Learning-rate schedule** (paper, Eq. 3), with `warmup_steps = 4000`:

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step^{-0.5}, step \cdot warmup_steps^{-1.5})$$

- 3.5 Batch size 64. Train for 20 epochs on a Colab or Kaggle GPU. After every epoch log the training loss, the dev loss and the current learning rate, and save the checkpoint with the lowest dev loss.
- 3.6 The dev set is used only for choosing the checkpoint; the test set is used once, at the very end.

2.4 Task 4 – Decode and convert to SQL

- 4.1 Implement **greedy decoding** and **beam search** (beam size 4) that stop at `</s>` or after 64 tokens.
- 4.2 Detokenise with `sp.decode` and parse the string back into WikiSQL's format: `{"sel": int, "agg": int, "conds": [[col, op, value], ...]}`, using the index lists `AGG_OPS` and `COND_OPS` from `data_prep.py`.
- 4.3 Write one JSON line per dev (or test) example, **in the same order as dev.jsonl**: `{"query": {...}}`. If the output cannot be parsed, write `{"error": "parse"}` for that line instead. Never skip a line.

- 4.4 Also write a function that turns the parsed query back into readable SQL with the real column names (for the front end and the blog).

2.5 Task 5 – Evaluate and analyse

- 5.1 Run the official evaluator (Section 3) on dev for greedy and beam output, and on test once with your final decoding choice.
- 5.2 **Component accuracy** on dev: the percentage of examples where the predicted `sel` column is right, where `agg` is right, and where the `WHERE` clause is right (same set of conditions, order ignored).
- 5.3 **Attention map:** for one dev example, plot the decoder cross-attention weights of the last layer (averaged over heads), generated tokens $\times$ source tokens. Check whether the generated `<cK>` tokens attend to the matching column in the source.

2.6 Task 6 – Front end and communication

- 6.1 A web interface (Streamlit, Gradio, Flask, ...) where the user types a question and a comma-separated list of column names and receives the generated SQL with the real column names. It must call *your* trained model. Screenshot in the README.
- 6.2 **Medium blog** (800–1500 words): the task, how the Transformer was built, the results, the attention map, and failure cases. At least two figures.
- 6.3 **LinkedIn post** linking the blog and repository.
- 6.4 **GitHub repository:** public, reproducible README, meaningful commits from *both* members.

2.7 Required repository layout

<code>starter/</code>	the four given files and <code>check_starter.py</code> , unchanged
<code>model/attention.py</code>	scaled dot-product and multi-head attention
<code>model/layers.py</code>	feed-forward, encoder layer, decoder layer
<code>model/transformer.py</code>	full model and masks
<code>train.py</code>	Task 3
<code>decode.py</code>	greedy, beam search, parser, readable SQL
<code>evaluate_model.py</code>	writes prediction files, component accuracy, attention map
<code>results/</code>	prediction files, tables, figures, <code>samples.md</code>
<code>app/</code>	front end
<code>README.md</code>	how to run everything

3. Evaluation

3.1 Official metrics

Use the evaluator shipped with WikiSQL; do not write your own. From inside the WikiSQL folder:

```
1 python evaluate.py data/dev.jsonl data/dev.db ../results/dev-greedy.jsonl
```

It prints two numbers:

Logical-form accuracy	the predicted query equals the gold query (<code>sel</code> , <code>agg</code> and the set of conditions)
Execution accuracy	the predicted query, run on the real table in <code>dev.db</code> , returns the same result as the gold query

Also report the **parse-failure rate**: the percentage of lines you wrote as `{"error": "parse"}`.

Expected range

For reference, the original LSTM sequence-to-sequence baseline in the WikiSQL paper reached about 36% execution accuracy on test, and the best published models exceed 90% using pretrained encoders. A correctly implemented Transformer of this size, trained from scratch with the column-token framing, should land clearly above the LSTM baseline; execution accuracy is usually several points higher than logical-form accuracy. Warning signs: execution accuracy below 20% means a bug (check the masks first); a parse-failure rate above 5% means a decoding bug; execution accuracy above 95% means gold data is leaking into your predictions.

3.2 Correctness checks

Include the output of these checks in your README.

- **Causal mask**: change the last token of a decoder input; the decoder outputs at all earlier positions must not change.
- **Padding mask**: add extra `<pad>` tokens to a source; the model's output must not change.
- **Attention rows sum to 1** over the unmasked positions.
- **Weight sharing**: the output projection weight and the embedding weight are the same tensor (`is` check).
- **Learning rate**: plot the schedule for the first 20,000 steps; it must rise linearly for 4000 steps, then decay as $step^{-0.5}$.
- **Gold round-trip**: feed the *gold* targets of dev through your parser and the official evaluator; execution accuracy must be above 99%.

4. Results

Present the following in `results/` and in the blog, using these layouts.

4.1 Table 1 – Data

	Train	Dev	Test
Pairs			
Mean / max source length (tokens)			
Mean / max target length (tokens)			
Pairs dropped as too long		–	–

4.2 Table 2 – Model and training

Trainable parameters
Epochs trained / best epoch
Best dev loss
Training time and GPU

4.3 Table 3 – Official metrics

Split	Decoding	Logical form (%)	Execution (%)	Parse failures (%)
Dev	greedy			
Dev	beam (4)			
Test	----			

4.4 Table 4 – Component accuracy (dev)

sel column correct (%)
agg correct (%)
WHERE clause correct (%)

4.5 Figures

1. Positional-encoding heat-map (Task 1.4).
2. Training and dev loss per epoch.
3. Learning-rate schedule.
4. Cross-attention map for one dev example (Task 5.3).
5. Front-end screenshot.

4.6 Qualitative samples

Ten dev examples in `results/samples.md` and the blog: five correct, five wrong. Show the question, the gold SQL and your SQL with real column names. For each wrong one, name the failure (wrong column, wrong aggregation, missing or extra condition, wrong value, parse failure, ...).

Appendix: Starter Code

The starter code covers everything up to and including the embedding layer. It was tested with Python 3.11, PyTorch and SentencePiece. Put all files in a folder called `starter/`.

A.1 Environment

```
1 pip install torch sentencepiece records babel tqdm tabulate
```

`records`, `babel`, `tqdm` and `tabulate` are needed only by the official WikiSQL evaluator.

A.2 Getting the data

```
1 git clone https://github.com/salesforce/WikiSQL
2 cd WikiSQL
3 tar xvjf data.tar.bz2          # creates WikiSQL/data/
4 cd ..
```

Your folder should now contain `WikiSQL/` and `starter/` side by side. Run the starter scripts from inside `starter/` after creating a link to the data: `ln -s ../WikiSQL WikiSQL` (on Windows, copy the folder instead).

A.3 data_prep.py – load and preprocess

```
1 import json
2 from pathlib import Path
3
4 DATA_DIR = Path("WikiSQL/data")
5 AGG_OPS = ["", "MAX", "MIN", "COUNT", "SUM", "AVG"]
6 COND_OPS = ["=", ">", "<"]
7 MAX_COLS = 64          # column tokens <c0> ... <c63>
8
9 def load_split(split):
10     """Return (examples, tables) for 'train', 'dev' or 'test'."""
11     tables = {}
12     with open(DATA_DIR / f"{split}.tables.jsonl", encoding="utf-8")
13         as f:
14         for line in f:
15             t = json.loads(line)
16             tables[t["id"]] = t
17     with open(DATA_DIR / f"{split}.jsonl", encoding="utf-8") as f:
18         examples = [json.loads(line) for line in f]
19     return examples, tables
20
21 def encode_source(question, header):
22     """question <sep> <c0> col name <c1> col name ..."""
23     cols = " ".join(f"<c{i}> {name}" for i, name in enumerate(header)
24                     )
25     return f"{question.strip()} <sep> {cols}".lower()
26
27 def encode_target(sql):
28     """{'sel', 'agg', 'conds'} -> 'select count <c3> where <c1> = kim
29     manners'"""
30     out = ["select"]
31     if sql["agg"]:
32         out.append(AGG_OPS[sql["agg"]].lower())
33     out.append(f"<c{sql['sel']}>")
```

```

31     for i, (col, op, val) in enumerate(sql["conds"]):
32         out += ["where" if i == 0 else "and", f"<c{col}>", COND_OPS[
            op], str(val)]
33     return " ".join(out).lower()
34
35 def build_pairs(split):
36     examples, tables = load_split(split)
37     pairs = []
38     for ex in examples:
39         header = tables[ex["table_id"]]["header"]
40         pairs.append({
41             "table_id": ex["table_id"],
42             "src": encode_source(ex["question"], header),
43             "tgt": encode_target(ex["sql"]),
44         })
45     return pairs
46
47 if __name__ == "__main__":
48     for split in ["train", "dev", "test"]:
49         pairs = build_pairs(split)
50         with open(f"{split}_pairs.jsonl", "w", encoding="utf-8") as f:
51             :
52             for p in pairs:
53                 f.write(json.dumps(p, ensure_ascii=False) + "\n")
54         print(f"{split}: {len(pairs)} pairs")
55     print(pairs[0]["src"]); print(pairs[0]["tgt"])

```

A.4 tokenizer.py – shared BPE vocabulary

```

1  import json
2  import sentencepiece as spm
3  from data_prep import MAX_COLS
4
5  PAD_ID, UNK_ID, BOS_ID, EOS_ID = 0, 1, 2, 3
6  SPECIAL = ["<sep>"] + [f"<c{i}>" for i in range(MAX_COLS)]
7
8  def read_pairs(path):
9      with open(path, encoding="utf-8") as f:
10         return [json.loads(line) for line in f]
11
12 def train_tokenizer(train_path="train_pairs.jsonl", vocab_size=8000):
13     pairs = read_pairs(train_path)
14     with open("spm_corpus.txt", "w", encoding="utf-8") as f:
15         for p in pairs:
16             vocabulary for
17             f.write(p["src"] + "\n")
18             as in
19             f.write(p["tgt"] + "\n")
20             Transformer
21             spm.SentencePieceTrainer.train(
22                 input="spm_corpus.txt", model_prefix="sql_sp",
23                 vocab_size=vocab_size, model_type="bpe",
24                 character_coverage=1.0,
25                 user_defined_symbols=SPECIAL,
26                 # never split <sep>, <
27                 c0>, ...
28                 pad_id=PAD_ID, unk_id=UNK_ID, bos_id=BOS_ID, eos_id=EOS_ID,
29             )
30     return spm.SentencePieceProcessor(model_file="sql_sp.model")

```

```

26
27 if __name__ == "__main__":
28     sp = train_tokenizer()
29     p = read_pairs("dev_pairs.jsonl")[0]
30     print(sp.encode(p["src"], out_type=str))
31     print(sp.encode(p["tgt"], out_type=str))
32     print(sp.decode(sp.encode(p["tgt"]))) == p["tgt"])

```

A.5 dataset.py – tensors and batches

```

1 import torch
2 from torch.utils.data import Dataset, DataLoader
3 from tokenizer import read_pairs, PAD_ID, BOS_ID, EOS_ID
4
5 class SQLDataset(Dataset):
6     """Each item: (source ids, target ids). Target = <s> ... </s>.
7     train=True drops over-long pairs.
8     train=False keeps EVERY row in file order (needed for evaluation)
9     ."""
9     def __init__(self, path, sp, train=True, max_src=160, max_tgt=64):
10         :
11         self.items, skipped = [], 0
12         for p in read_pairs(path):
13             src = sp.encode(p["src"]) + [EOS_ID]
14             tgt = [BOS_ID] + sp.encode(p["tgt"]) + [EOS_ID]
15             if train and (len(src) > max_src or len(tgt) > max_tgt):
16                 skipped += 1
17                 continue
18             self.items.append((src, tgt))
19             print(f"{path}: kept {len(self.items)}, skipped {skipped}")
20
21     def __len__(self):
22         return len(self.items)
23
24     def __getitem__(self, i):
25         return self.items[i]
26
27 def collate(batch):
28     srcs, tgts = zip(*batch)
29     S, T = max(map(len, srcs)), max(map(len, tgts))
30     src = torch.full((len(batch), S), PAD_ID, dtype=torch.long)
31     tgt = torch.full((len(batch), T), PAD_ID, dtype=torch.long)
32     for i, (s, t) in enumerate(zip(srcs, tgts)):
33         src[i, :len(s)] = torch.tensor(s)
34         tgt[i, :len(t)] = torch.tensor(t)
35     return src, tgt # (B, S), (B, T)
36
37 def make_loader(path, sp, train, batch_size=64):
38     return DataLoader(SQLDataset(path, sp, train=train),
39                       batch_size=batch_size, shuffle=train,
40                       collate_fn=collate)

```

A.6 embeddings.py – embeddings and positional encoding

```

1 import math
2 import torch
3 import torch.nn as nn

```

```

4
5 class TokenEmbedding(nn.Module):
6     """Section 3.4: embeddings are multiplied by sqrt(d_model).
7     One instance is shared by the encoder, the decoder and
8     (optionally) the final output projection."""
9     def __init__(self, vocab_size, d_model, pad_id=0):
10         super().__init__()
11         self.emb = nn.Embedding(vocab_size, d_model, padding_idx=
12             pad_id)
13         self.scale = math.sqrt(d_model)
14
15         def forward(self, ids):
16             # (B, L) -> (B, L, d_model)
17             return self.emb(ids) * self.scale
18
19 class PositionalEncoding(nn.Module):
20     """Section 3.5: fixed sinusoidal encodings.
21     PE(pos, 2i) = sin(pos / 10000^(2i/d_model))
22     PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))"""
23     def __init__(self, d_model, max_len=512, dropout=0.1):
24         super().__init__()
25         self.dropout = nn.Dropout(dropout)
26         pos = torch.arange(max_len).unsqueeze(1) # (L, 1)
27         div = torch.exp(torch.arange(0, d_model, 2)
28             * (-math.log(10000.0) / d_model)) # (d/2,)
29         pe = torch.zeros(max_len, d_model)
30         pe[:, 0::2] = torch.sin(pos * div)
31         pe[:, 1::2] = torch.cos(pos * div)
32         self.register_buffer("pe", pe.unsqueeze(0)) # (1, L, d)
33
34         def forward(self, x):
35             # x: (B, L, d_model)
36             return self.dropout(x + self.pe[:, : x.size(1)])
37
38 class InputLayer(nn.Module):
39     """token ids -> scaled embedding + positional encoding -> dropout"""
40     def __init__(self, token_emb, d_model, max_len=512, dropout=0.1):
41         super().__init__()
42         self.tok = token_emb
43         self.pos = PositionalEncoding(d_model, max_len, dropout)
44
45         def forward(self, ids):
46             return self.pos(self.tok(ids))

```

A.7 check_starter.py – where your code begins

```

1 import sentencepiece as spm
2 from dataset import make_loader
3 from embeddings import TokenEmbedding, InputLayer
4 from tokenizer import PAD_ID
5
6 sp = spm.SentencePieceProcessor(model_file="sql_sp.model")
7 train_dl = make_loader("train_pairs.jsonl", sp, train=True)
8 dev_dl = make_loader("dev_pairs.jsonl", sp, train=False)
9

```

```

10 src, tgt = next(iter(train_dl))
11 print("src", tuple(src.shape), "tgt", tuple(tgt.shape))
12
13 d_model = 256
14 shared = TokenEmbedding(sp.get_piece_size(), d_model, PAD_ID)
15 enc_in = InputLayer(shared, d_model)      # encoder input
16 dec_in = InputLayer(shared, d_model)      # decoder input (same
      weights)
17
18 x = enc_in(src)                          # (B, S, 256) -> goes into YOUR encoder
19 y = dec_in(tgt[:, :-1])                  # (B, T-1, 256) -> goes into YOUR decoder
20 print("encoder input", tuple(x.shape), "decoder input", tuple(y.shape
    ))

```

Expected output (batch lengths vary because batches are shuffled):

```

1 train_pairs.jsonl: kept 56336, skipped 19
2 dev_pairs.jsonl: kept 8421, skipped 0
3 src (64, 123) tgt (64, 31)
4 encoder input (64, 123, 256) decoder input (64, 30, 256)

```

The two tensors `x` and `y` are the inputs to the encoder and decoder you build in Task 2.

Good luck. Build something a non-programmer could actually use.